دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

Computer Architecture

Central Processing Unit

Afarghadan@aut.ac.ir

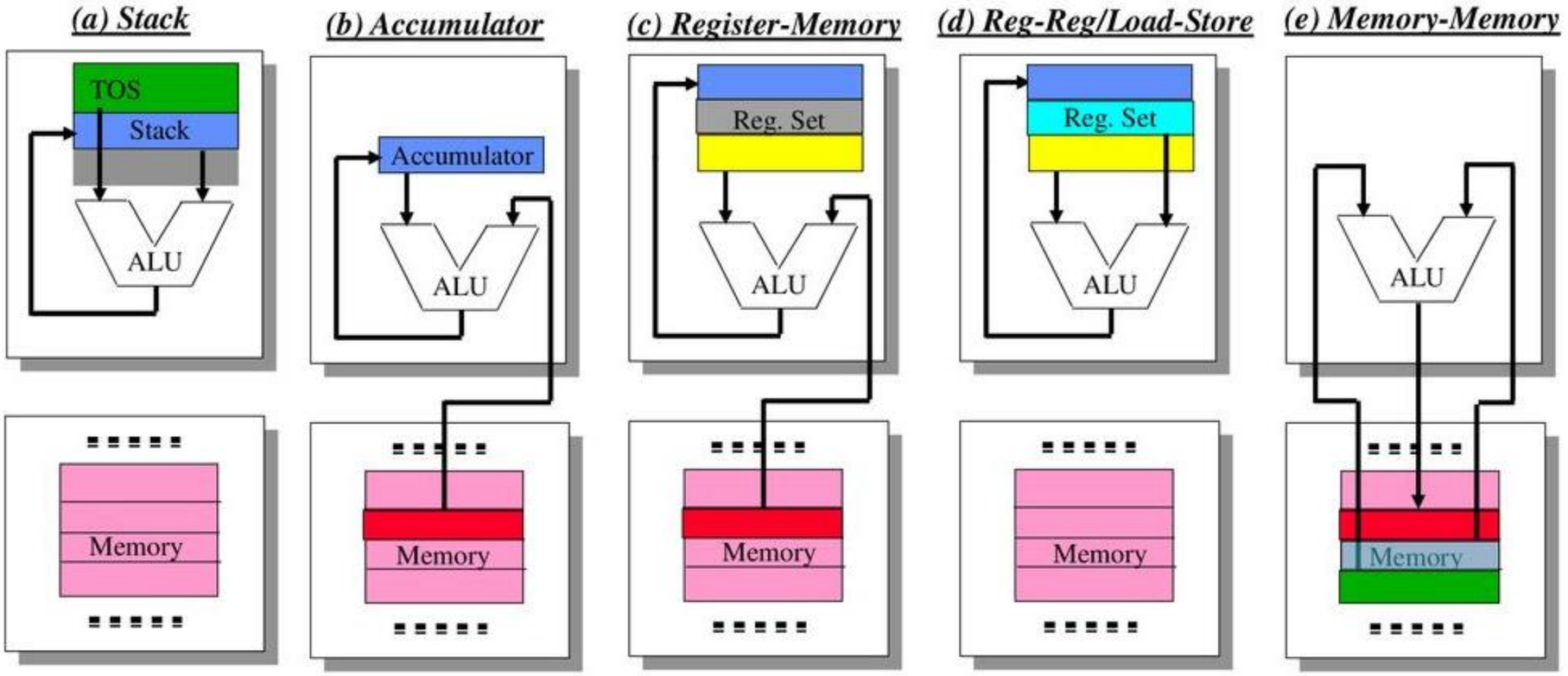
عظیم فرقدان

Lecture Overview

1. **Central Processing Unit**
2. **Stack**
3. **Operands' Placement**
4. **Addressing Modes**
5. **RISC vs CISC**
6. **Input-Output Organization**
7. **Direct Memory Access**

Where are operands?

C←A+B



Push A
Push B
Add
Pop C

Load A
Add B
Store C

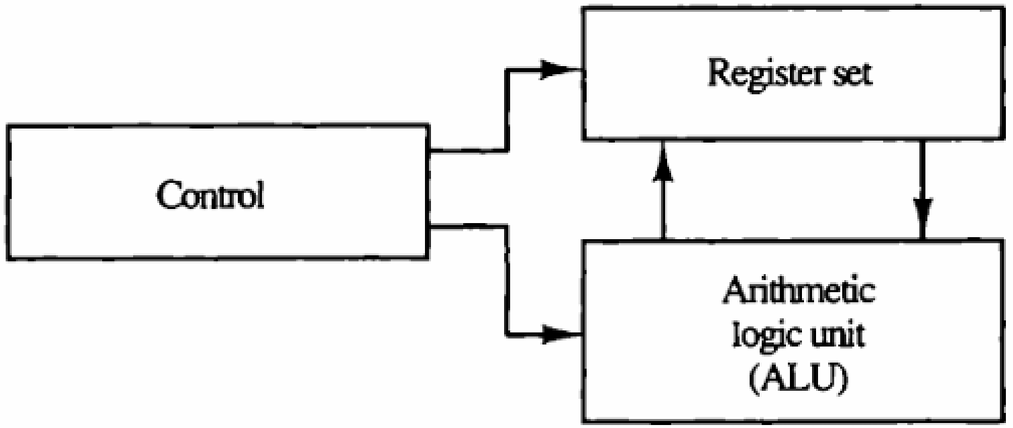
Load R1,A
Add R1,B
Store C,R1

Load R1,A
Load R2,B
Add R3,R1,R2
Store C,R3

Add C,A,B
or Add A,B

Introduction

- CPU is the part of the computer that performs data-processing operations.
- CPU has three major parts:
  - The register set: stores intermediate data used during execution
  - The arithmetic logic unit (ALU) performs the microoperations
  - The control unit: supervises the transfer of information between registers and instructs the ALU

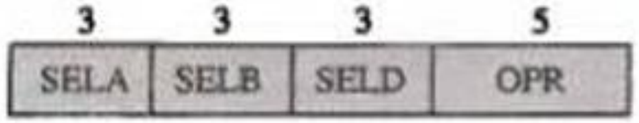


Major components of CPU.

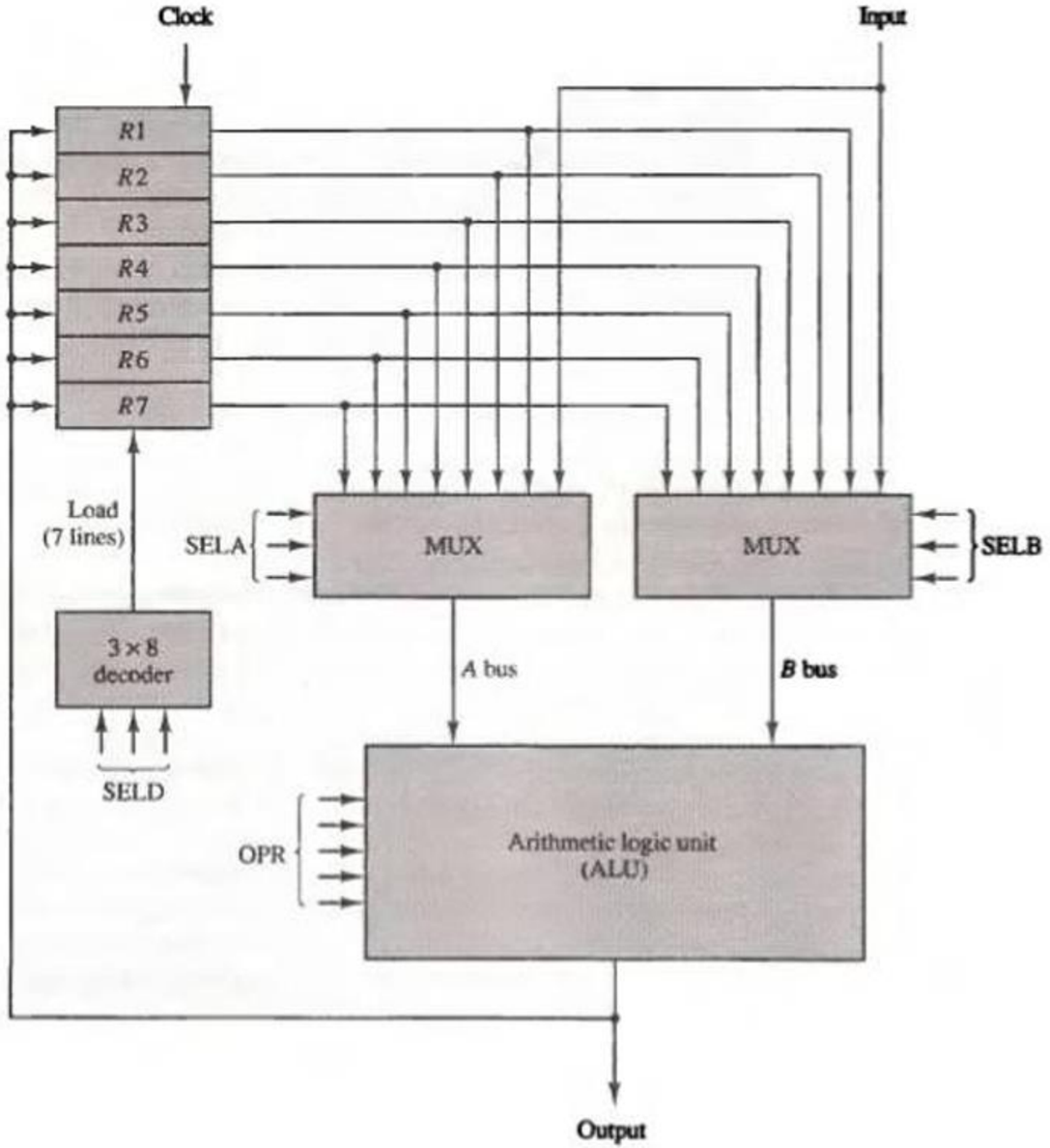
General Register Organization

- A large number of registers in CPU
  - More efficient, more convenient and faster
  - Connected through a common bus system

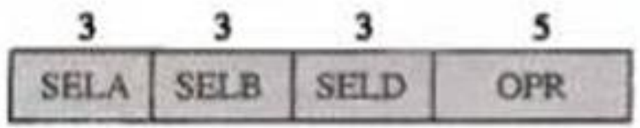
- In the following example, there are 14 binary selection inputs in the unit and their combined value specifies a **control word**



(b) Control word



Control Word



(b) Control word

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	*R1*	*R1*	*R1*
010	*R2*	*R2*	*R2*
011	*R3*	*R3*	*R3*
100	*R4*	*R4*	*R4*
101	*R5*	*R5*	*R5*
110	*R6*	*R6*	*R6*
111	*R7*	*R7*	*R7*

OPR Select	Operation	Symbol
00000	Transfer *A*	TSFA
00001	Increment *A*	INCA
00010	Add *A* + *B*	ADD
00101	Subtract *A* − *B*	SUB
00110	Decrement *A*	DECA
01000	AND *A* and *B*	AND
01010	OR *A* and *B*	OR
01100	XOR *A* and *B*	XOR
01110	Complement *A*	COMA
10000	Shift right *A*	SHRA
11000	Shift left *A*	SHLA

Example of Microoperations

$R1 \leftarrow R2 - R3$

Field:	SELA	SELB	SELD	OPR
Symbol:	R2	R3	R1	SUB
Control word:	010	011	001	00101

Microoperation	SELA	SELB	SELD	OPR	Control Word
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	—	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	—	R7	TSFA	001 000 111 00000
Output $\leftarrow R2$	R2	—	None	TSFA	010 000 000 00000
Output $\leftarrow$ Input	Input	—	None	TSFA	000 000 000 00000
$R4 \leftarrow$ shl $R4$	R4	—	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100

Micropogrammed Control Unit

- Many microoperations can be generated in the CPU. The most efficient way to generate control words with a large number of bits is to store them in a memory unit.

- A **memory unit** that stores control words is called a control memory.

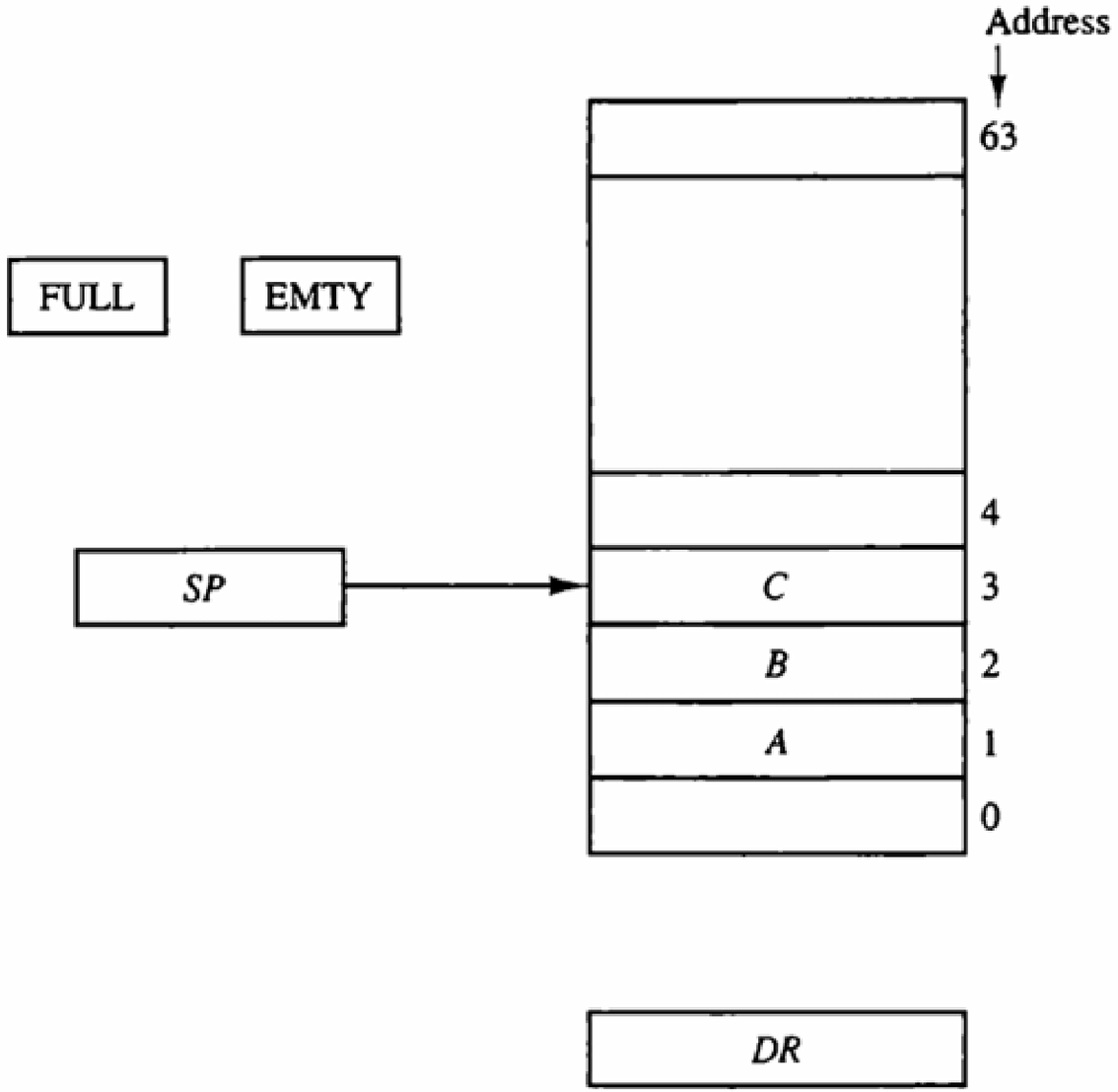
- By reading consecutive control words from memory, it’s possible to initiate a sequence of microoperations for the CPU.

- This type of control is referred to as **microprogrammed control**.

Stack Organization

- Stack is a last-in, first-out (**LIFO**) list.
- The stack is a **memory unit** with an address register.
- The register that holds the address for the stack is called a **stack pointer (SP)** which always points out to the top of the stack.
- **Push**:
  - It’s an **insert** operation
  - The result of pushing a new item on top
- **Pop**:
  - It’s a **deletion** operation
  - The result of removing one item so that the stack pops up.
- These operations are simulated by **incrementing** or **decrementing** the stack pointer register.

Stack Operations



Push

$SP \leftarrow SP + 1$	Increment stack pointer
$M[SP] \leftarrow DR$	Write item on top of the stack
If $(SP = 0)$ then $(FULL \leftarrow 1)$	Check if stack is full
$EMTY \leftarrow 0$	Mark the stack not empty

Pop

$DR \leftarrow M[SP]$	Read item from the top of stack
$SP \leftarrow SP - 1$	Decrement stack pointer
If $(SP = 0)$ then $(EMTY \leftarrow 1)$	Check if stack is empty
$FULL \leftarrow 0$	Mark the stack not full

A new item is inserted with the push operation as follows:

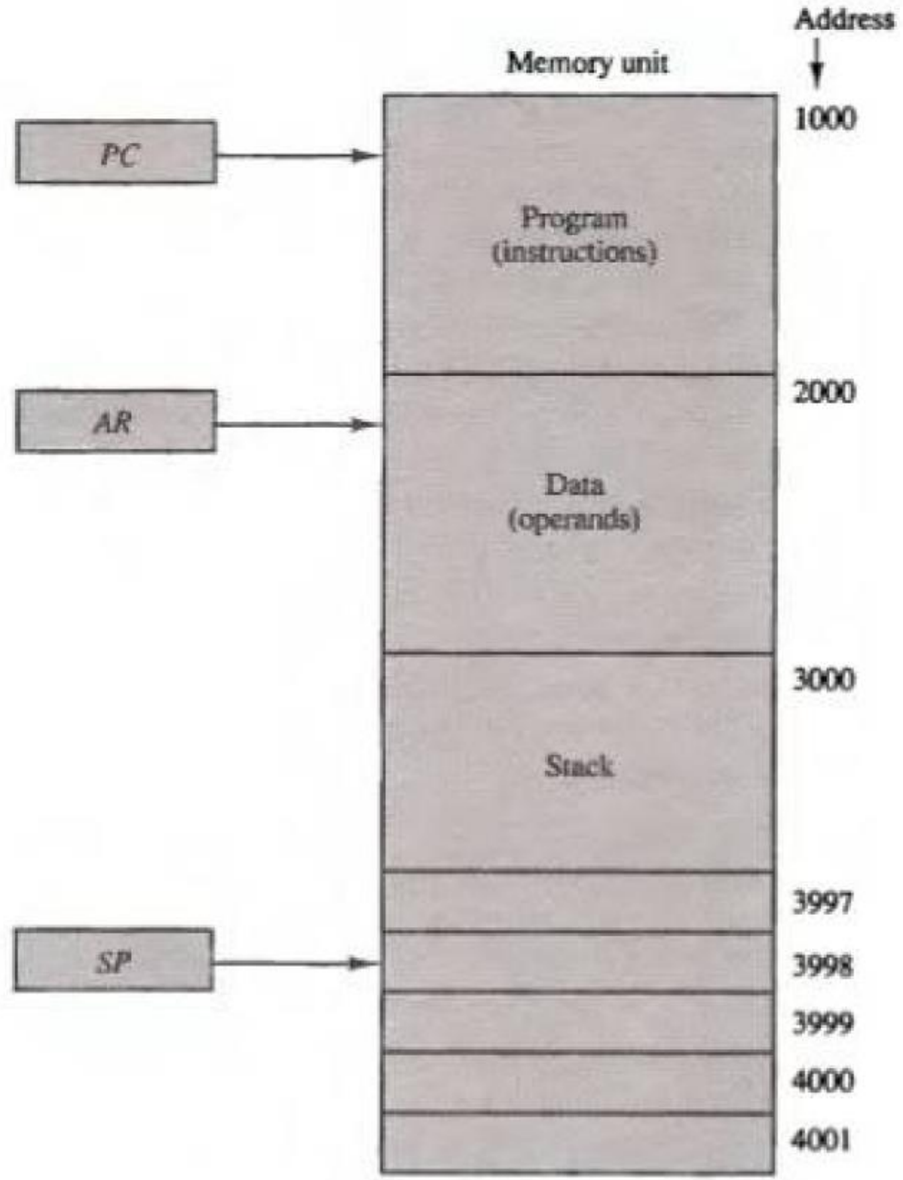
$$SP \leftarrow SP - 1$$

$$M[SP] \leftarrow DR$$

A new item is deleted with a pop operation as follows:

$$DR \leftarrow M[SP]$$

$$SP \leftarrow SP + 1$$



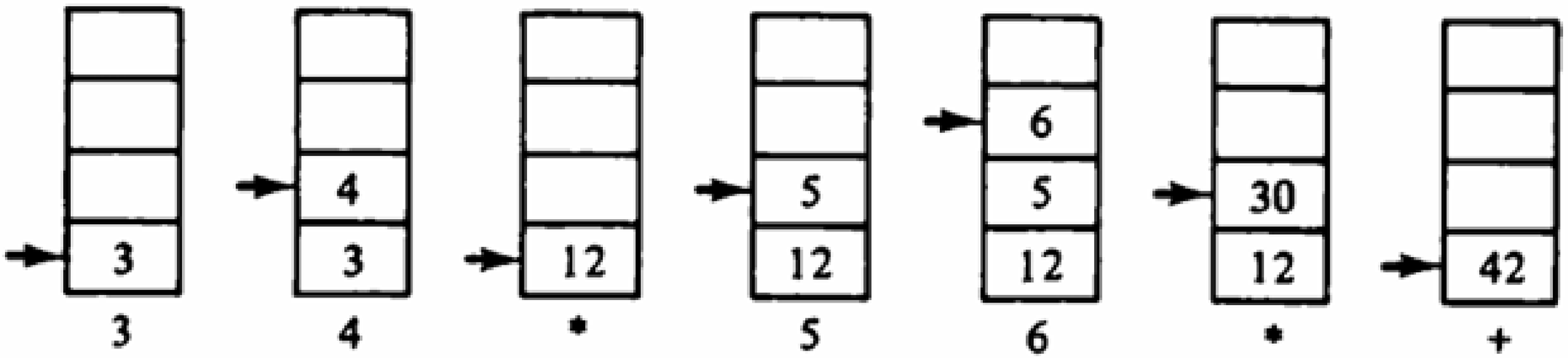
Stack Arithmetic

Infix notation	Prefix / Polish notation	Postfix / reverse Polish notation
A + B	**+ A B**	**A B +**

$$A * B + C * D \rightarrow A\ B * C\ D * +$$

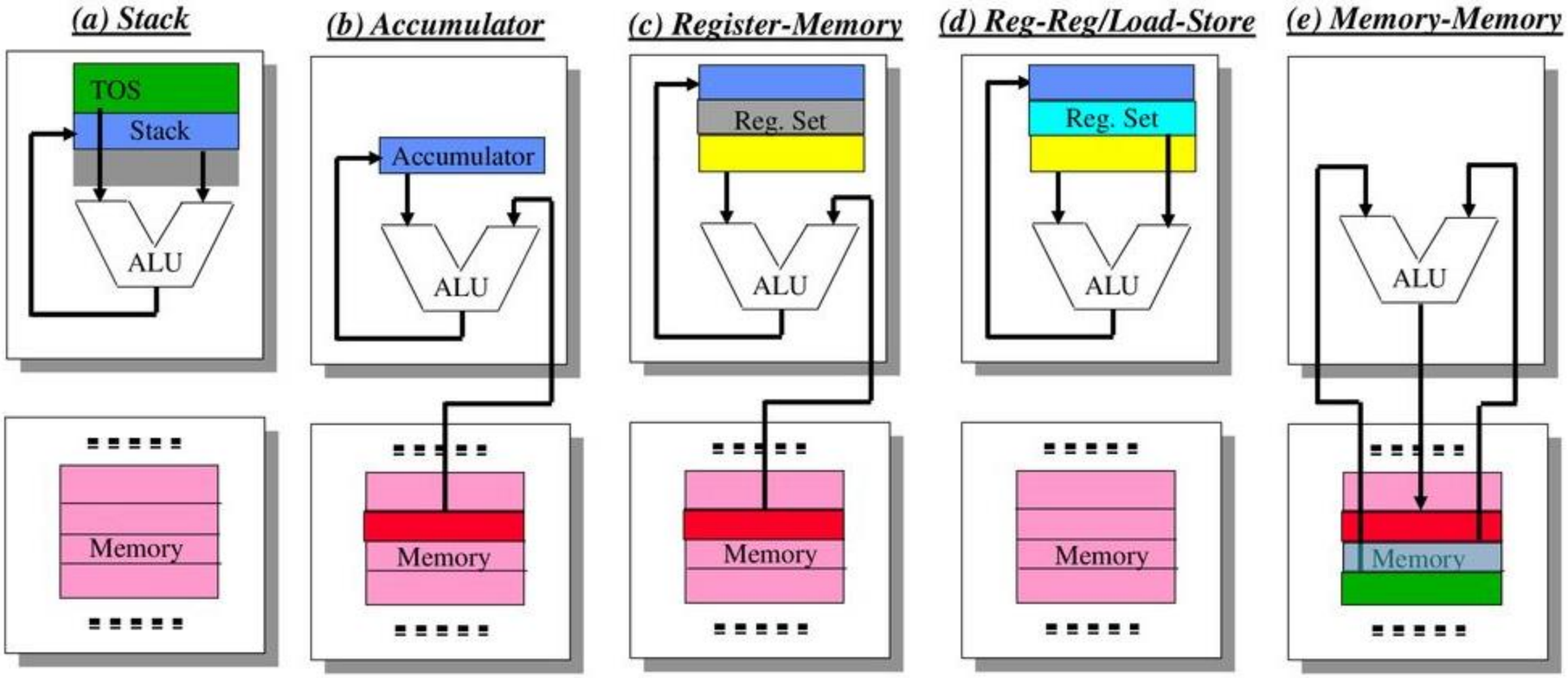
$$(A + B) * [C * (D + E) + F] \rightarrow A\ B + D\ E + C * F + *$$

$$(3 * 4) + (5 * 6) \rightarrow (5 * 6)$$



Where are operands?

C←A+B



Push A
Push B
Add
Pop C

Load A
Add B
Store C

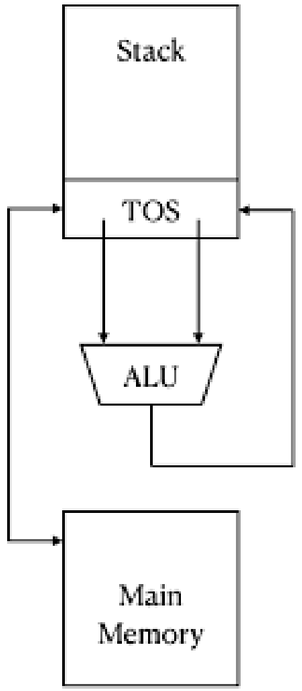
Load R1,A
Add R1,B
Store C,R1

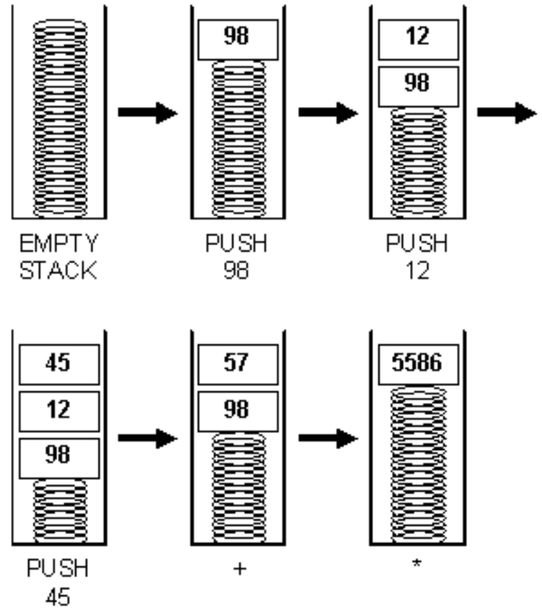
Load R1,A
Load R2,B
Add R3,R1,R2
Store C,R3

Add C,A,B
or Add A,B

Stack Machine

- **0-operand ISA**
  - ALU operations (add, sub, and) don’t need any operands
- Push
  - Loads mem into 1st reg (“top of stack”),
- Pop
  - Does reverse
- Add, Sub, Mul, and etc.
  - Combines contents of first two regs



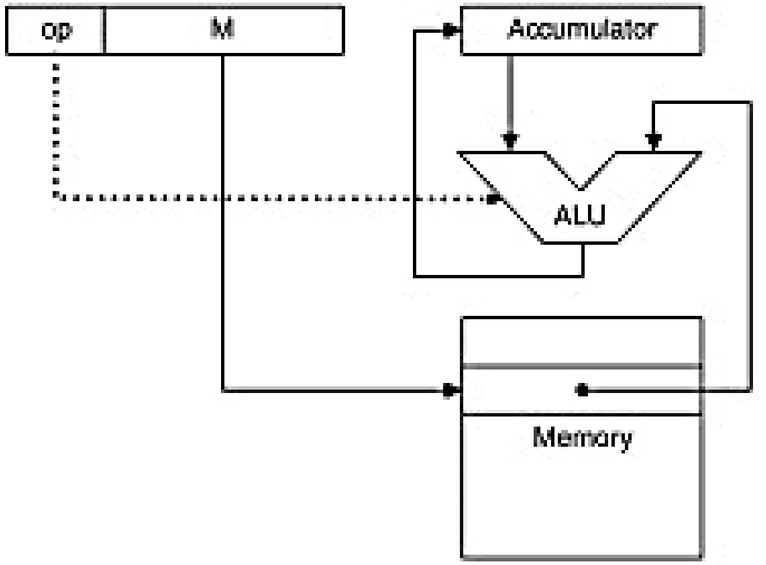


Accumulator Machine

- **1-operand** ISA

- Only 1 register
  - Called **accumulator**
  - Stores intermediate arithmetic & logic results

- **Instructions** include
  - Store
  - Load
  - acc ← acc + mem

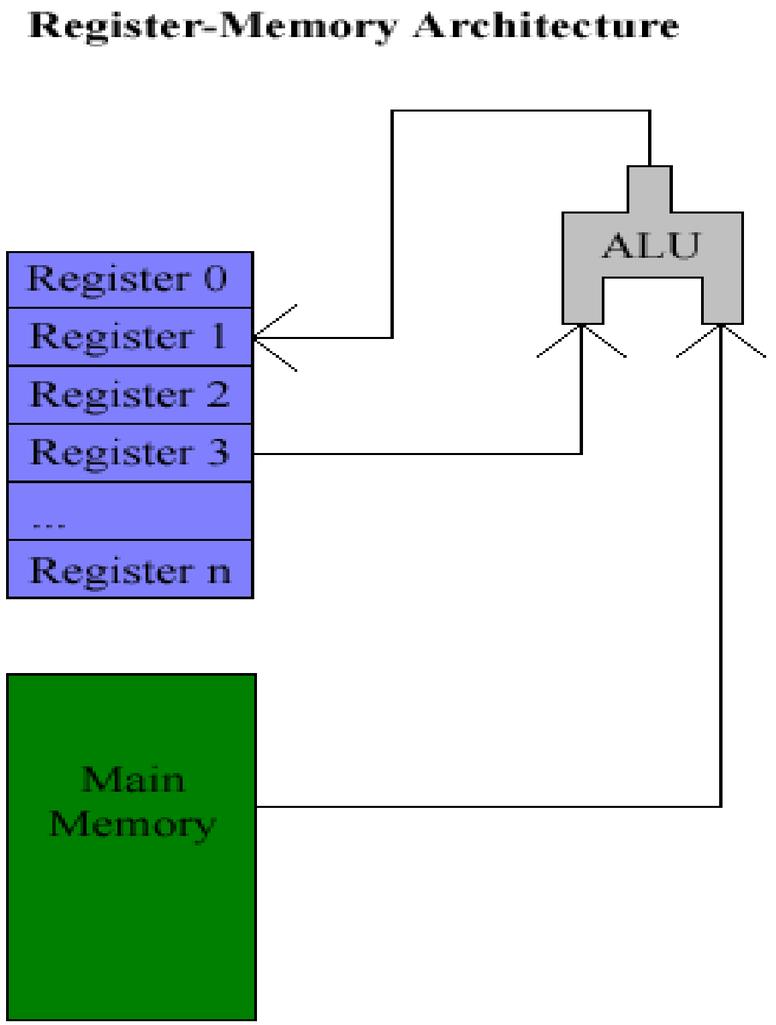


(b) Accumulator

Register-Memory Machine

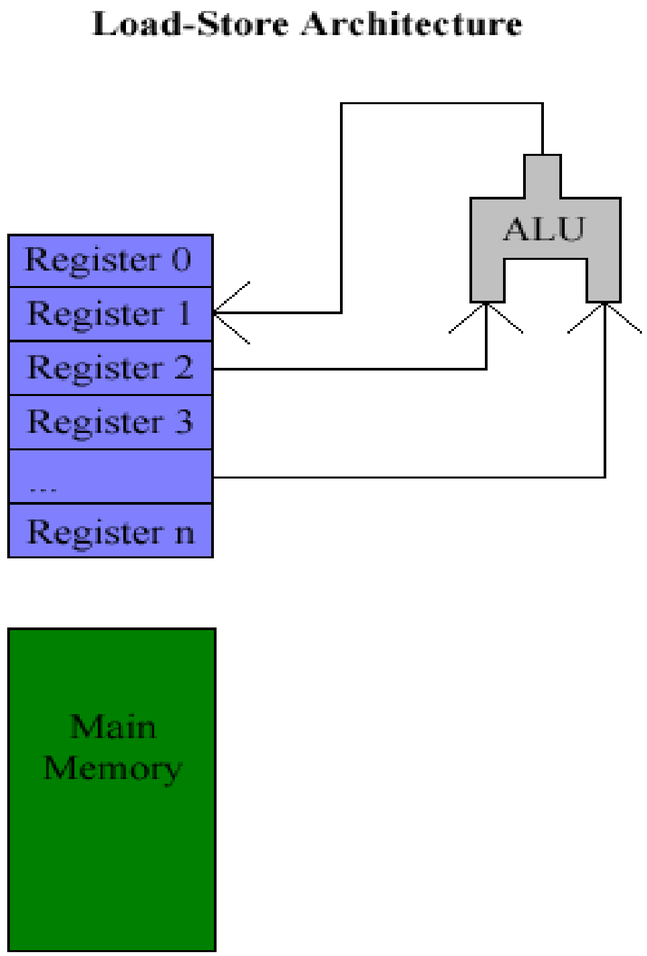
- Operands
  - **Register or memory**

- Arithmetic instructions can use data in registers and/or memory



Register-Register Machine

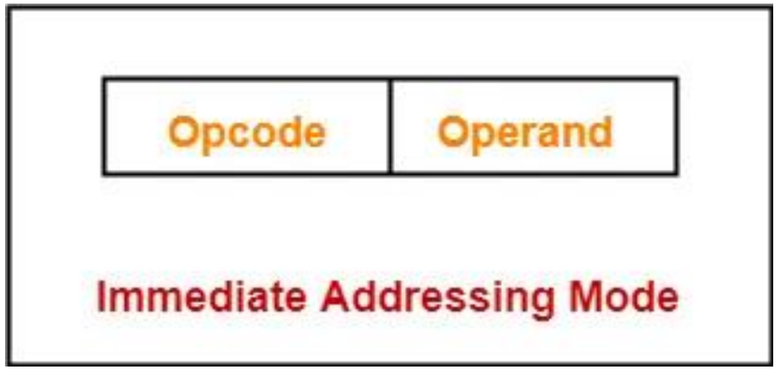
- Also known as **load-store machine**
- Operands
  - **Register**
- Arithmetic instructions can use data in registers
- **No operation on memory**
  - Only read and write instruction (load and store)



Addressing Modes

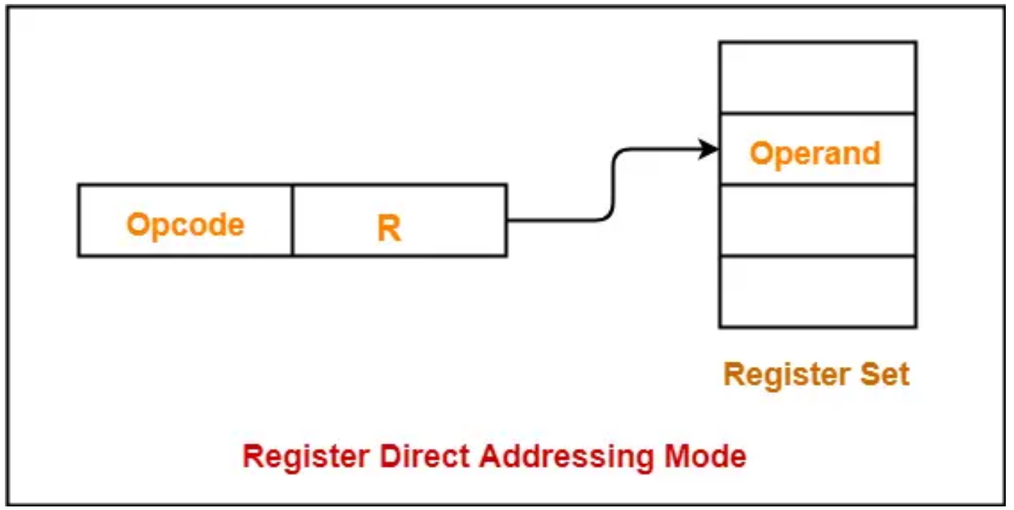
- **Implied (Implicit) Addressing Mode**
  - The definition of the instruction itself specify the operands implicitly.
  - Example: Complement ACC

- **Immediate Mode**
  - The operand is specified in the instruction explicitly.
  - Example: Add R1, 105

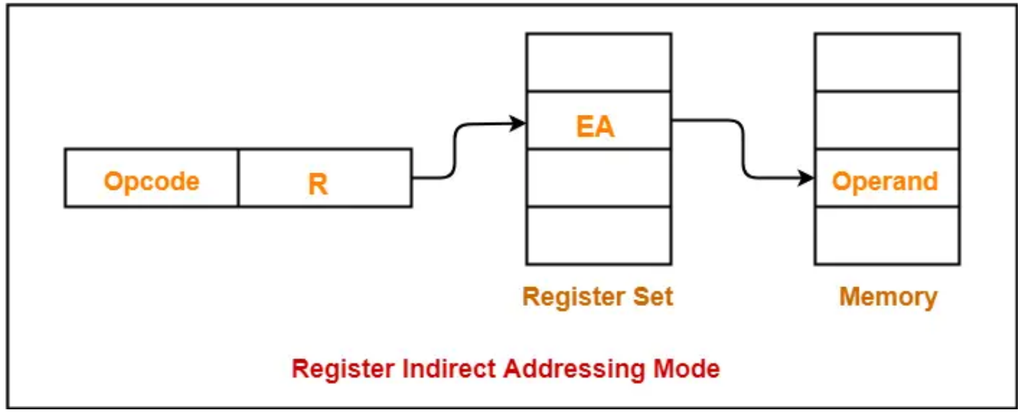


Addressing Modes

- **Register Mode**
  - The address field refers to a register that contains the operand.
  - Example: Add R1, R2, R3

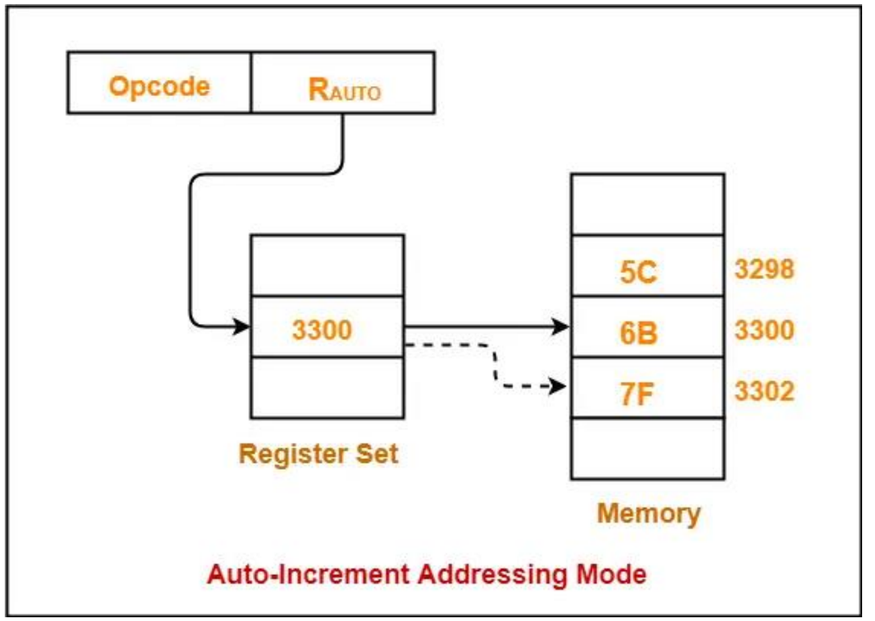


- **Register Indirect Mode**
  - The address field of the instruction refers to a CPU register that contains the effective address of the operand.
  - Example: Add R1, R2, [R3]

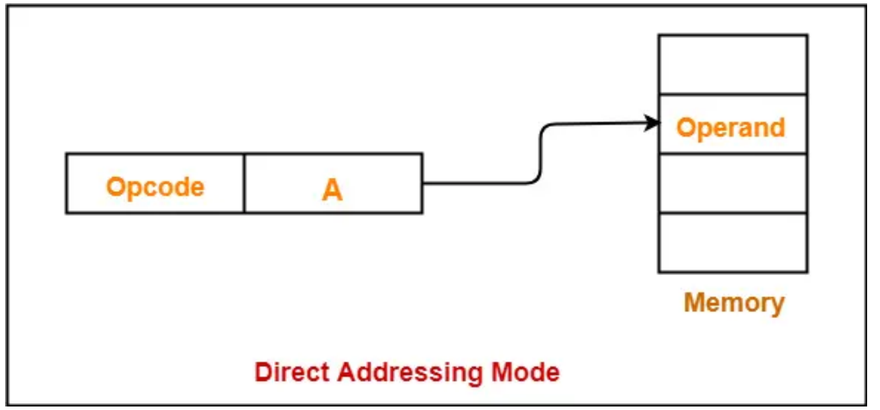


Addressing Modes

- **Auto increment/decrement Mode:**
  - After accessing the operand, the content of the register is incremented or decremented by step size ‘d’
  - Example: Add R1, R2, [R3+4]

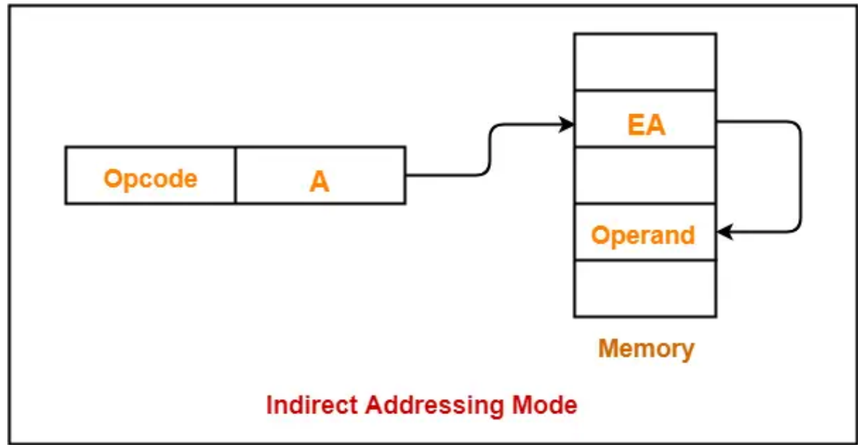


- **Direct Mode:**
  - The address field of the instruction contains the effective address of the operand.
  - Example: AC ← AC + [X]

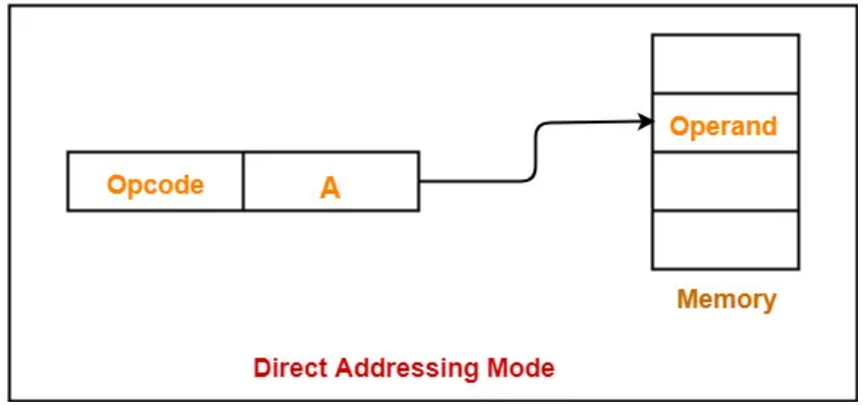


Addressing Modes

- **Indirect Mode**
  - The address field of the instruction specifies the address of memory location that contains the effective address of the operand.
  - AC ← AC + [[X]]

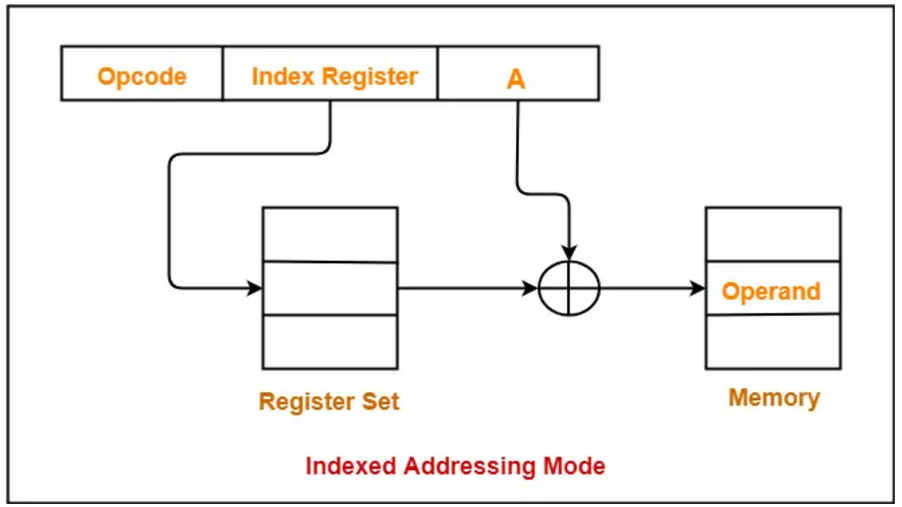


- **Relative Mode**
  - The address field of the instruction refers to a CPU register that contains the operand.
  - Example: AC ← AC + [R]

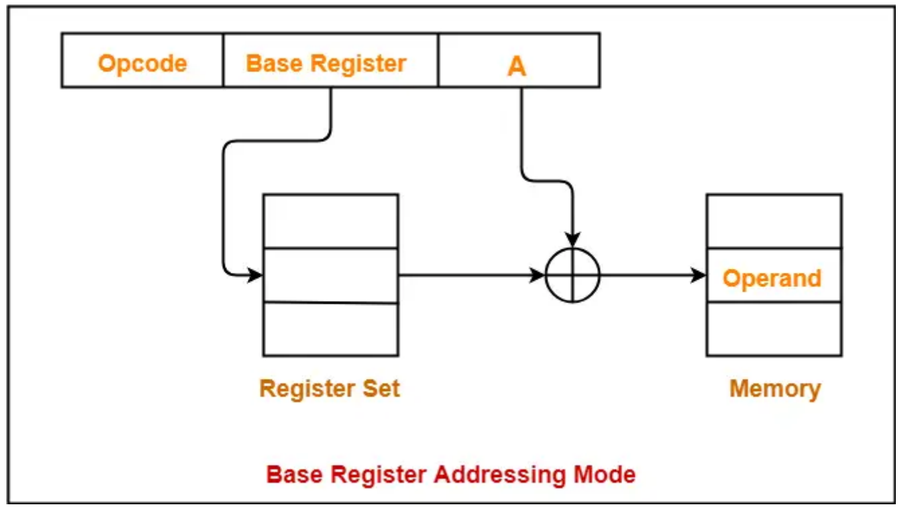


Addressing Modes

- **Indexed Addressing Mode**
  - Effective address of the operand is obtained by adding the content of index register with the address part of the instruction
  - Example: Add R1, R2, [R3+Const.]



- **Base Register Mode**
  - Effective address of the operand is obtained by adding the content of base register with the address part of the instruction.
  - Example: LW R1, 100(R3)



RISC and CISC

- **Complex Instruction Set Computer (CISC)**
  - A computer with a large number of instructions is classified as a complex instruction set computer, abbreviated CISC
  - This includes more complex instructions e.g. MUL (multiplication), DIV (division) and MOD (modulo)

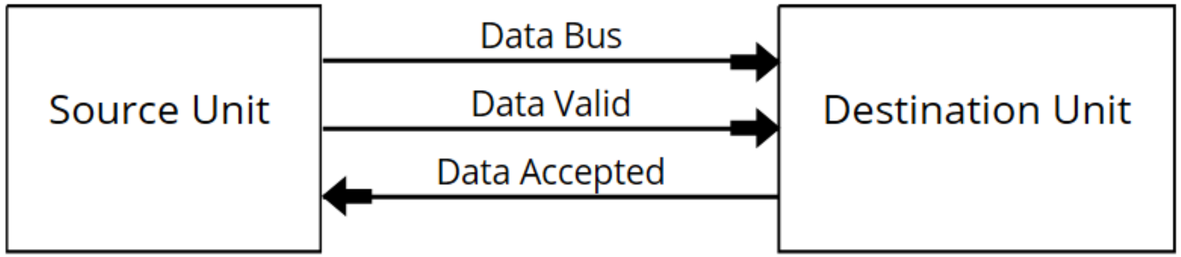
- **Reduced Instruction Set Computer (RISC)**
  - a number of computer designers recommended that computers use fewer instructions with simple constructs so they can be exe-cuted much faster within the CPU without having to use memory as often

RISC vs. CISC

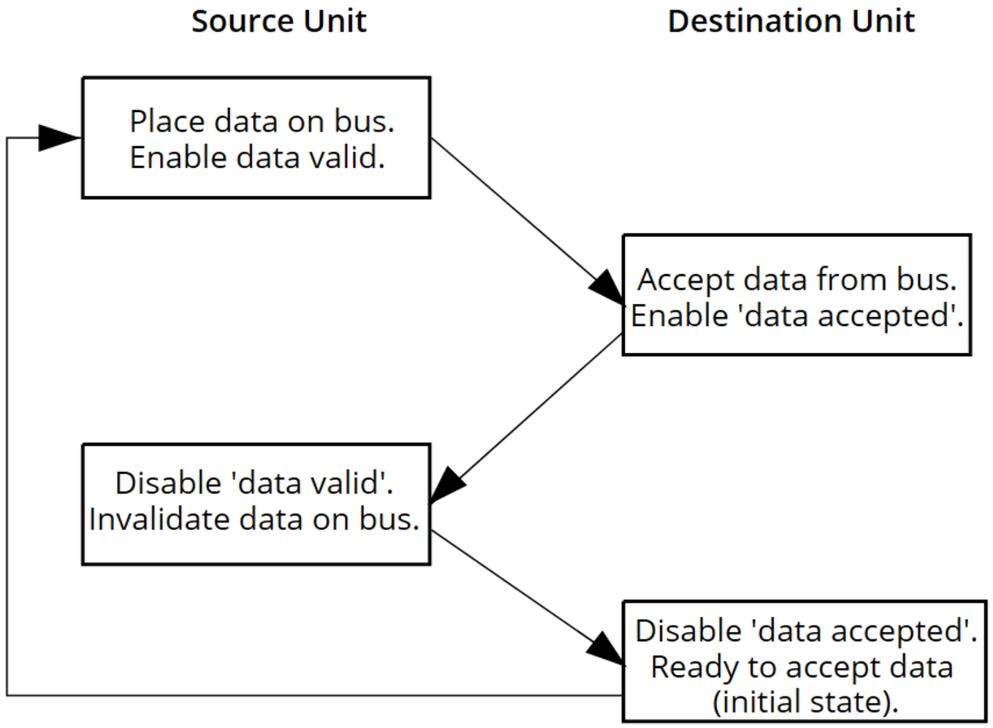
- **RISC**: Reduced Instruction Set Computer
- **CISC:** Complex Instruction Set Computer

	CISC	RISC
Number of instructions	Few	A lot
Addressing mode	A lot and with variety	Few
Instruction length	Variable	Constant
Direct memory access	Few	A lot
Hardware complexity	Complex	Simple
Number of registers	Few	A lot
Cycles for instr. execution	A lot and variable	One
Clock frequency	High	Low
Code size	Small	Big

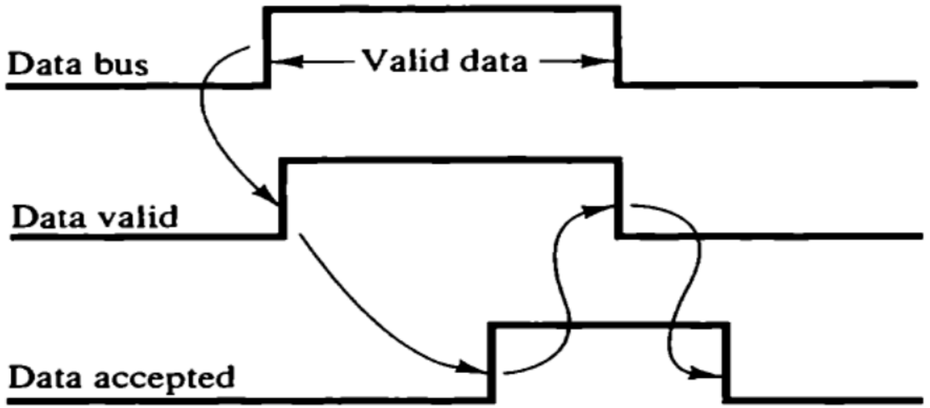
- **Asynchronous Data Transfer**
  - Handshaking
    - Source-initiated



Block Diagram



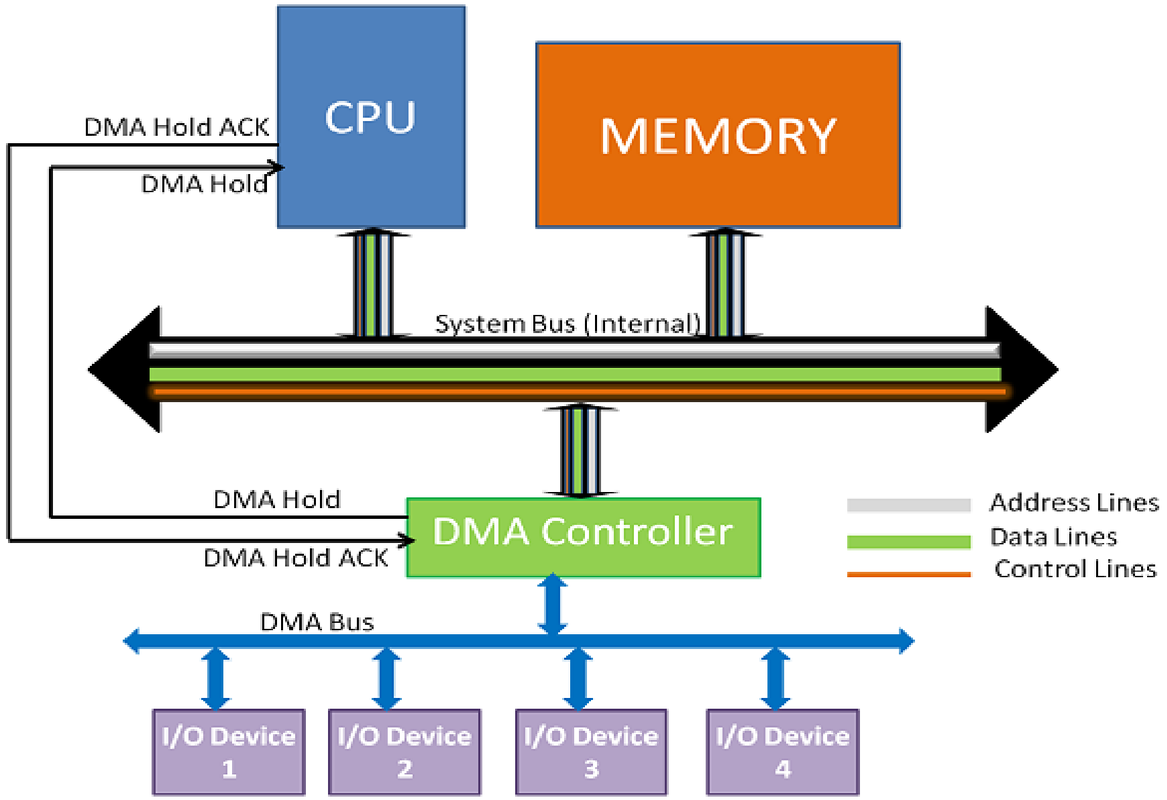
Event Diagram



(b) Timing diagram

Direct Memory Access (DMA)

- A method that allows an input/output (I/O) device to send or receive data directly to or from the main memory
  - Bypassing the CPU to speed up memory operations
  - Is managed by a chip known as a DMA controller (DMAC)



Direct Memory Access (DMA)

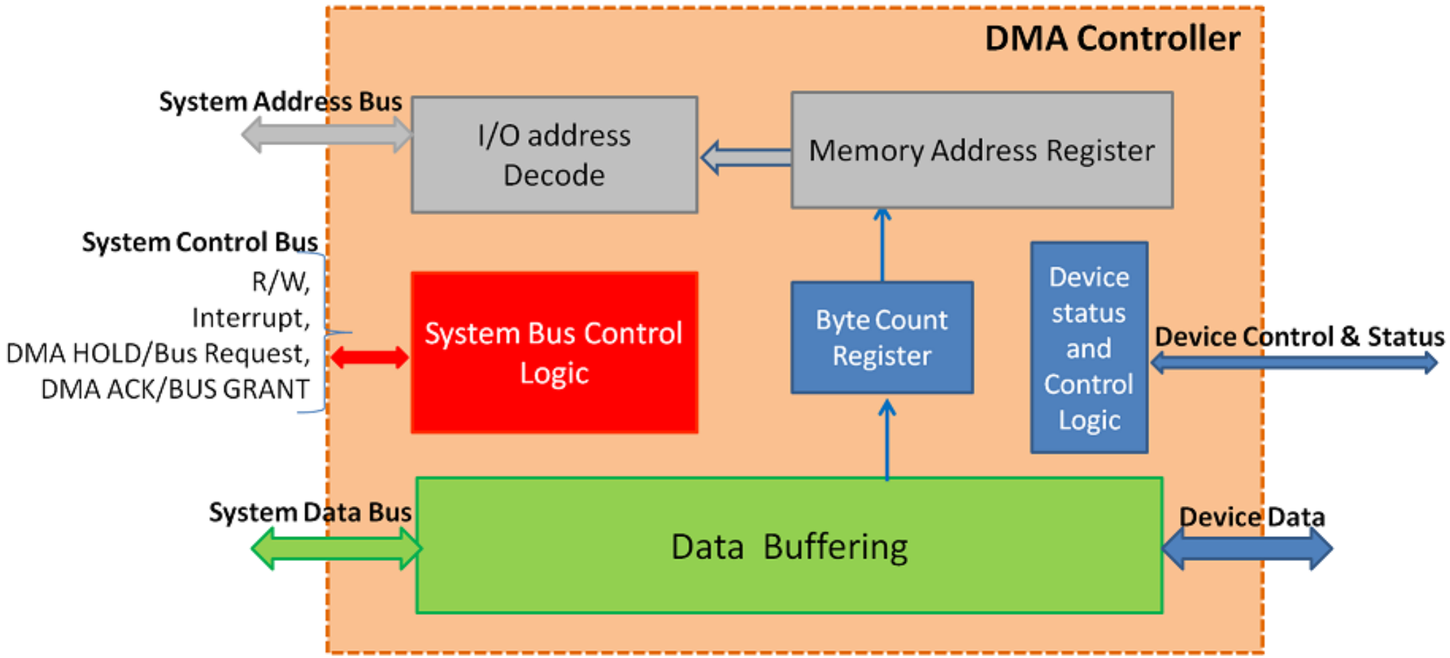
- **data transfer steps between I/O device and Memory using DMA:**
  - CPU delegates the responsibility of data transfer to DMA by sending the required details.
  - The I/O controller initiates the necessary actions with the device and requests DMAC when it is ready with data.
  - DMAC raises HOLD to CPU signaling its intention to take the System bus for data transfer.
  - At the end of the current machine cycle, the CPU disengages itself from the system bus. Then, the CPU responds with a HOLD ACKNOWLEDGE signal to DMAC, indicating that the system bus is available for use.
  - DMAC places the memory address on the address bus, the location at which data transfer is to take place.
  - A read or write signal is then generated by the DMAC, and the I/O device either generates or latches the data. Then DMA transfers data to memory.
  - A register is used as a byte count, decremented for each byte transferred to memory.
  - Increment the memory address by the number of bytes transferred and generate new memory address for the next memory cycle.
  - Upon the byte count reaches zero, the DMAC generates an Interrupt to CPU.

Direct Memory Access (DMA)

- **DMA controller transfers the data in three modes:**
  - **Burst Mode**: Here, once the DMA controller gains the charge of the system bus, then it releases the system bus only after completion of data transfer. Till then the CPU has to wait for the system buses.
  - **Cycle Stealing Mode:** In this mode, the DMA controller forces the CPU to stop its operation and relinquish the control over the bus for a short term to DMA controller. After the transfer of every byte, the DMA controller releases the bus and then again requests for the system bus. In this way, the DMA controller steals the clock cycle for transferring every byte.
  - **Transparent Mode:** Here, the DMA controller takes the charge of system bus only if the processor does not require the system bus.

Direct Memory Access Controller (DMAC)

- The **bus control logic** generates R/W signal and Interrupt. The bus signals are DMAHOLD or Bus Request, DMA HOLD ACK or BUS GRANT.
- **Byte Count Register** holds and tracks the byte transfer taking place with Memory. CPU sets the value. It is decremented for every byte transferred and the Memory address register is updated to show the next location for data transfer. When the byte count becomes zero, it indicates **End of Data Transfer**. This status is used for generating an **interrupt signal** to CPU. **Buffered Data** Transfer takes place at the device end.



Thanks